POSTGRESQL · EQUiDADOS · VERSÃO ATUAL · 08 NOV 2026

Scripts de criação e inserção do banco

DDL do banco e scripts de carga — reunidos em um único PDF

Conteúdo e ordem de execução

1. **Criação do esquema:** tabelas, restrições e índices básicos do `arquivo.sql`.
2. **Objetos da Etapa 04:** dois índices avançados e duas visões, extraídos do script da etapa.
3. **Inserção/carga:** scripts Python que leem as coleções do MongoDB e inserem/atualizam os registros no PostgreSQL/Neon.

Como os dados são inseridos

As fontes reais de unidades, equipes e Censo são carregadas por `mongo_para_neon.py`. Os dados de atendimentos e a referência territorial fictícios/simulados são carregados por `mongo_para_neon_mock.py`. Os scripts criam as tabelas necessárias antes da carga e usam comandos parametrizados com `INSERT ... ON CONFLICT` para inserir ou atualizar pelas chaves primárias.

> **Configuração:** a execução requer `MONGO_URI` e `NEON_DATABASE_URL` definidos no arquivo local `.env`. Credenciais não fazem parte deste documento. As tabelas com prefixo `mock_` contêm apenas dados fictícios/simulados; não representam pacientes nem atendimentos reais.

Arquivos-fonte incluídos

- `arquivo.sql` — DDL de criação (somente a seção de criação).
- `Etapa_04_Consultas_Avancadas.sql` — índices e visões.
- `src/transform/mongo_para_neon.py` — criação e carga das tabelas reais.
- `src/transform/mongo_para_neon_mock.py` — criação e carga das tabelas simuladas.

01 | Criação do esquema base

DDL PostgreSQL: tabelas, chaves, restrições, comentários e índices básicos.

Arquivo de origem: `arquivo.sql`

```sql
-- EquiDados: esquema PostgreSQL/Neon e consultas da interface.
-- Os dados das tabelas mock_ sao ficticios/simulados.
-- Este script cria a estrutura e executa apenas consultas SELECT ao final.

BEGIN;

CREATE TABLE IF NOT EXISTS unidades_saude (
    cnes                      TEXT PRIMARY KEY,
    nome_fantasia             TEXT,
    nome_empresarial          TEXT,
    bairro                    TEXT,
    distrito_sanitario_codigo TEXT,
    distrito_sanitario_nome   TEXT,
    endereco                  TEXT,
    cep                       TEXT,
    latitude                  DOUBLE PRECISION,
    longitude                 DOUBLE PRECISION,
    tipo_estabelecimento      TEXT
);

CREATE TABLE IF NOT EXISTS equipes_saude (
    codigo_equipe    TEXT PRIMARY KEY,
    nome_equipe      TEXT,
    tipo_equipe      TEXT,
    cnes_unidade     TEXT REFERENCES unidades_saude(cnes),
    data_ativacao    TEXT,
    data_desativacao TEXT
);

CREATE TABLE IF NOT EXISTS censo_raca_cor (
    raca_cor  TEXT PRIMARY KEY,
    populacao BIGINT,
    ano       INTEGER
);

CREATE TABLE IF NOT EXISTS censo_deficiencia (
    tipo_dificuldade TEXT PRIMARY KEY,
    populacao        BIGINT,
    ano              INTEGER,
    e_total          BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS mock_pec_atendimentos (
    id_atendimento                   INTEGER PRIMARY KEY,
    codigo_equipe                    TEXT REFERENCES equipes_saude(codigo_equipe),
    cnes_unidade                     TEXT REFERENCES unidades_saude(cnes),
    distrito_sanitario_codigo        TEXT,
    data_atendimento                 DATE,
    tipo_atendimento                 TEXT,
    profissional_tipo                TEXT,
    turno_atendimento                TEXT,
    idade                            INTEGER,
    raca_cor                         TEXT,
    deficiencia_tipo                 TEXT,
    deseja_informar_orientacao_sexual TEXT,
    orientacao_sexual                TEXT,
    deseja_informar_identidade_genero TEXT,
    identidade_genero                TEXT,
    nome_social                      TEXT,
    CONSTRAINT ck_mock_atendimento_idade
        CHECK (idade IS NULL OR idade BETWEEN 0 AND 120)
);

ALTER TABLE mock_pec_atendimentos
    ADD COLUMN IF NOT EXISTS turno_atendimento TEXT;

CREATE TABLE IF NOT EXISTS mock_populacao_negra_distrito (
    distrito_sanitario_codigo    TEXT PRIMARY KEY,
    populacao_negra_pct_esperado NUMERIC
        CHECK (
            populacao_negra_pct_esperado IS NULL
            OR populacao_negra_pct_esperado BETWEEN 0 AND 1
        )
);

COMMENT ON TABLE mock_pec_atendimentos IS
```

```
    'DADOS FICTICIOS/SIMULADOS. Nao representa pacientes ou atendimentos reais.';

COMMENT ON TABLE mock_populacao_negra_distrito IS
    'DADOS FICTICIOS/SIMULADOS. Referencia ilustrativa, nao representa '
    'a distribuicao real da populacao por distrito.';

CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_data
    ON mock_pec_atendimentos (data_atendimento);

CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_distrito
    ON mock_pec_atendimentos (distrito_sanitario_codigo);

CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_equipe
    ON mock_pec_atendimentos (codigo_equipe);

COMMIT;
```

02 | Índices e visões da Etapa 04

DDL da etapa avançada; as consultas SELECT do arquivo original não fazem parte deste recorte.

Arquivo de origem: `Etapa_04_Consultas_Avancadas.sql`

```sql
-- ETAPA 04 - Consultas avancadas, visoes e indices
-- Banco: PostgreSQL / Neon
-- As tabelas mock_ contem dados ficticios/simulados.
--
-- Os indices atendem os filtros por equipe/data e data/distrito das
-- consultas de atividade recente e dos indicadores territoriais.
-- Execute este arquivo conectado ao banco EquiDados.

BEGIN;

CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_equipe_data_etapa04
    ON mock_pec_atendimentos (codigo_equipe, data_atendimento);

CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_data_distrito_etapa04
    ON mock_pec_atendimentos (data_atendimento, distrito_sanitario_codigo);

-- Visao 1: consolidado territorial de capacidade, atendimentos e equidade.
-- Justificativa semantica: a gestao pode comparar, por distrito, a estrutura
-- de unidades e equipes com os atendimentos simulados, completude de raca/cor
-- e a referencia territorial ilustrativa. Os agregados sao calculados antes
-- dos JOINs para evitar multiplicar contagens entre unidades e atendimentos.
-- A visao realiza cinco LEFT JOINs entre agregados de dominio relacionados.
CREATE OR REPLACE VIEW vw_equidade_distrital AS
WITH distritos AS (
    SELECT distrito_sanitario_codigo
    FROM unidades_saude
    WHERE distrito_sanitario_codigo IS NOT NULL
    UNION
    SELECT distrito_sanitario_codigo
    FROM mock_pec_atendimentos
    WHERE distrito_sanitario_codigo IS NOT NULL
    UNION
    SELECT distrito_sanitario_codigo
    FROM mock_populacao_negra_distrito
),
unidades AS (
    SELECT distrito_sanitario_codigo,
           COUNT(DISTINCT cnes) AS total_unidades
    FROM unidades_saude
    WHERE distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
),
equipes AS (
    SELECT u.distrito_sanitario_codigo,
           COUNT(DISTINCT e.codigo_equipe) AS total_equipes
    FROM equipes_saude e
    JOIN unidades_saude u ON u.cnes = e.cnes_unidade
    WHERE u.distrito_sanitario_codigo IS NOT NULL
    GROUP BY u.distrito_sanitario_codigo
),
atendimentos AS (
    SELECT distrito_sanitario_codigo,
           COUNT(*) AS total_atendimentos_simulados,
           COUNT(raca_cor) AS registros_com_raca_cor,
           COUNT(*) FILTER (
               WHERE raca_cor IN ('Preta', 'Parda')
           ) AS registros_pretos_ou_pardos,
           COUNT(*) FILTER (
               WHERE idade >= 10
           ) AS atendimentos_com_10_anos_ou_mais
    FROM mock_pec_atendimentos
    WHERE distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
),
atendimentos_recentes AS (
    SELECT distrito_sanitario_codigo,
           COUNT(*) AS atendimentos_ultimos_90_dias
    FROM mock_pec_atendimentos
    WHERE data_atendimento >= CURRENT_DATE - INTERVAL '90 days'
      AND data_atendimento < CURRENT_DATE + INTERVAL '1 day'
      AND distrito_sanitario_codigo IS NOT NULL
    GROUP BY distrito_sanitario_codigo
)
SELECT d.distrito_sanitario_codigo,
       COALESCE(u.total_unidades, 0) AS total_unidades,
       COALESCE(e.total_equipes, 0) AS total_equipes,
```

```
        COALESCE(a.total_atendimentos_simulados, 0)
            AS total_atendimentos_simulados,
        COALESCE(ar.atendimentos_ultimos_90_dias, 0)
            AS atendimentos_ultimos_90_dias,
        COALESCE(a.registros_com_raca_cor, 0) AS registros_com_raca_cor,
        ROUND(
            100.0 * COALESCE(a.registros_com_raca_cor, 0)
                / NULLIF(a.total_atendimentos_simulados, 0),
            1
        ) AS completude_raca_cor_pct,
        ROUND(
            100.0 * a.registros_pretos_ou_pardos
                / NULLIF(a.registros_com_raca_cor, 0),
            1
        ) AS percentual_preto_ou_pardo_observado,
        ROUND(100.0 * p.populacao_negra_pct_esperado, 1)
            AS percentual_preto_ou_pardo_referencia_simulada,
        ROUND(
            100.0 * p.populacao_negra_pct_esperado
                - 100.0 * a.registros_pretos_ou_pardos
                    / NULLIF(a.registros_com_raca_cor, 0),
            1
        ) AS diferenca_referencia_observado_pontos_percentuais,
        COALESCE(a.atendimentos_com_10_anos_ou_mais, 0)
            AS atendimentos_com_10_anos_ou_mais
FROM distritos d
LEFT JOIN unidades u
    ON u.distrito_sanitario_codigo = d.distrito_sanitario_codigo
LEFT JOIN equipes e
    ON e.distrito_sanitario_codigo = d.distrito_sanitario_codigo
LEFT JOIN atendimentos a
    ON a.distrito_sanitario_codigo = d.distrito_sanitario_codigo
LEFT JOIN atendimentos_recentes ar
    ON ar.distrito_sanitario_codigo = d.distrito_sanitario_codigo
LEFT JOIN mock_populacao_negra_distrito p
    ON p.distrito_sanitario_codigo = d.distrito_sanitario_codigo;

COMMENT ON VIEW vw_equidade_distrital IS
    'Consolida estrutura de saude, atendimentos simulados e indicadores '
    'territoriais de completude/equidade por distrito. A referencia e '
    'ilustrativa, nao representa estimativa oficial.';

-- Visao 2: completude e volume por equipe, unidade e distrito.
-- Justificativa semantica: equipes de gestao podem priorizar apoio e
-- capacitacao a partir do volume e da completude dos registros. A referencia
-- distrital serve de contexto, sem classificar desempenho individual.
-- A visao relaciona equipe, unidade, atendimentos agregados e referencia
-- distrital (tres LEFT JOINs).
CREATE OR REPLACE VIEW vw_completude_por_equipe AS
WITH atendimentos_equipe AS (
    SELECT codigo_equipe,
           COUNT(*) AS total_atendimentos_simulados,
           COUNT(raca_cor) AS registros_com_raca_cor,
           COUNT(deficiencia_tipo) AS registros_com_deficiencia,
           COUNT(*) FILTER (
               WHERE idade >= 10
           ) AS atendimentos_com_10_anos_ou_mais,
           COUNT(orientacao_sexual) FILTER (
               WHERE idade >= 10
           ) AS registros_com_orientacao_sexual,
           COUNT(identidade_genero) FILTER (
               WHERE idade >= 10
           ) AS registros_com_identidade_genero,
           MAX(data_atendimento) AS data_ultimo_atendimento
    FROM mock_pec_atendimentos
    WHERE codigo_equipe IS NOT NULL
    GROUP BY codigo_equipe
)
SELECT e.codigo_equipe,
       e.nome_equipe,
       e.tipo_equipe,
       u.cnes AS cnes_unidade,
       COALESCE(u.nome_fantasia, u.nome_empresarial) AS unidade,
       u.distrito_sanitario_codigo,
       COALESCE(a.total_atendimentos_simulados, 0)
           AS total_atendimentos_simulados,
       a.data_ultimo_atendimento,
       ROUND(
           100.0 * a.registros_com_raca_cor
               / NULLIF(a.total_atendimentos_simulados, 0),
           1
       ) AS completude_raca_cor_pct,
```

```
        ROUND(
            100.0 * a.registros_com_deficiencia
                / NULLIF(a.total_atendimentos_simulados, 0),
            1
        ) AS completude_deficiencia_pct,
        ROUND(
            100.0 * a.registros_com_orientacao_sexual
                / NULLIF(a.atendimentos_com_10_anos_ou_mais, 0),
            1
        ) AS completude_orientacao_sexual_10_mais_pct,
        ROUND(
            100.0 * a.registros_com_identidade_genero
                / NULLIF(a.atendimentos_com_10_anos_ou_mais, 0),
            1
        ) AS completude_identidade_genero_10_mais_pct,
        ROUND(100.0 * p.populacao_negra_pct_esperado, 1)
            AS percentual_negro_referencia_distrital_simulada
FROM equipes_saude e
LEFT JOIN unidades_saude u
    ON u.cnes = e.cnes_unidade
LEFT JOIN atendimentos_equipe a
    ON a.codigo_equipe = e.codigo_equipe
LEFT JOIN mock_populacao_negra_distrito p
    ON p.distrito_sanitario_codigo = u.distrito_sanitario_codigo;

COMMENT ON VIEW vw_completude_por_equipe IS
    'Completude agregada de registros ficticios por equipe, com contexto da '
    'unidade e referencia territorial simulada. Nao deve ser usada para '
    'avaliar individualmente profissionais.';

COMMIT;
```

03 | Inserção dos dados reais

Script Python completo: MongoDB → PostgreSQL/Neon, tabelas CNES e Censo.

Arquivo de origem: `src\transform\mongo_para_neon.py`

```python
import os

from dotenv import load_dotenv
from pymongo import MongoClient
from sqlalchemy import create_engine, text

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
NEON_DATABASE_URL = os.getenv("NEON_DATABASE_URL")
NOME_BANCO_MONGO = "bronze_equidade_saude"

DDL_TABELAS = """
CREATE TABLE IF NOT EXISTS unidades_saude (
    cnes                    TEXT PRIMARY KEY,
    nome_fantasia           TEXT,
    nome_empresarial        TEXT,
    bairro                  TEXT,
    distrito_sanitario_codigo TEXT,
    distrito_sanitario_nome TEXT,
    endereco                TEXT,
    cep                     TEXT,
    latitude                DOUBLE PRECISION,
    longitude               DOUBLE PRECISION,
    tipo_estabelecimento    TEXT
);

CREATE TABLE IF NOT EXISTS equipes_saude (
    codigo_equipe   TEXT PRIMARY KEY,
    nome_equipe     TEXT,
    tipo_equipe     TEXT,
    cnes_unidade    TEXT REFERENCES unidades_saude(cnes),
    data_ativacao   TEXT,
    data_desativacao TEXT
);

CREATE TABLE IF NOT EXISTS censo_raca_cor (
    raca_cor    TEXT PRIMARY KEY,
    populacao   BIGINT,
    ano         INTEGER
);

CREATE TABLE IF NOT EXISTS censo_deficiencia (
    tipo_dificuldade    TEXT PRIMARY KEY,
    populacao           BIGINT,
    ano                 INTEGER,
    e_total             BOOLEAN NOT NULL DEFAULT FALSE
);
"""


def criar_tabelas(engine):
    with engine.begin() as conexao:
        conexao.execute(text(DDL_TABELAS))
    print("Tabelas criadas.")


def transformar_unidades_saude(colecao):
    linhas = []
    for doc in colecao.find():
        linhas.append({
            "cnes": doc.get("cnes"),
            "nome_fantasia": doc.get("noFantasia"),
            "nome_empresarial": doc.get("noEmpresarial"),
            "bairro": doc.get("bairro"),
            "distrito_sanitario_codigo": doc.get("coDistritoSanitario"),
            "distrito_sanitario_nome": doc.get("dsDistrito"),
            "endereco": doc.get("noLogradouro"),
            "cep": doc.get("cep"),
            "latitude": float(doc["nuLatitude"]) if doc.get("nuLatitude") else None,
            "longitude": float(doc["nuLongitude"]) if doc.get("nuLongitude") else None,
            "tipo_estabelecimento": doc.get("dsTipoEstabelecimento"),
        })
    return linhas


def transformar_equipes_saude(colecao):
```

```
    linhas = []
    for doc in colecao.find():
        linhas.append({
            "codigo_equipe": doc.get("coEquipe"),
            "nome_equipe": doc.get("nomeEquipe"),
            "tipo_equipe": doc.get("dsEquipe"),
            "cnes_unidade": doc.get("_cnes_unidade"),
            "data_ativacao": doc.get("dtAtivacao"),
            "data_desativacao": doc.get("dtDesativacao"),
        })
    return linhas


def transformar_censo_raca_cor(colecao):
    linhas = []
    for doc in colecao.find():
        if doc.get("NC") == "Nível Territorial (Código)":
            continue  # linha de cabeçalho da API SIDRA, não é dado
        if doc.get("D4N") == "Total":
            continue  # a linha "Total" não é uma categoria de raça/cor, pulamos
        linhas.append({
            "raca_cor": doc.get("D4N"),
            "populacao": int(doc.get("V")),
            "ano": int(doc.get("D3N")),
        })
    return linhas


def transformar_censo_deficiencia(colecao):
    linhas = []
    for doc in colecao.find():
        if doc.get("NC") == "Nível Territorial (Código)":
            continue  # linha de cabeçalho da API SIDRA, não é dado
        if doc.get("D2N") != "Pessoas de 2 anos ou mais de idade com deficiência":
            continue
        # as categorias não são mutuamente exclusivas (uma pessoa pode ter mais
        # de um tipo de dificuldade), por isso guardamos o "Total" oficial
        # como uma linha marcada, em vez de deixar quem consome a tabela somar
        # as categorias e chegar num número inflado.
        linhas.append({
            "tipo_dificuldade": doc.get("D4N"),
            "populacao": int(doc.get("V")),
            "ano": int(doc.get("D3N")),
            "e_total": doc.get("D4N") == "Total",
        })
    return linhas


def inserir_linhas(engine, tabela, linhas):
    if not linhas:
        print(f"  nenhuma linha para inserir em '{tabela}'")
        return
    colunas = list(linhas[0].keys())
    placeholders = ", ".join(f":{c}" for c in colunas)
    atualizacoes = ", ".join(
        f"{coluna} = EXCLUDED.{coluna}" for coluna in colunas[1:]
    )
    sql = text(
        f"INSERT INTO {tabela} ({', '.join(colunas)}) VALUES ({placeholders}) "
        f"ON CONFLICT ({colunas[0]}) DO UPDATE SET {atualizacoes}"
    )
    with engine.begin() as conexao:
        conexao.execute(sql, linhas)
    print(f"  {len(linhas)} linha(s) inserida(s) em '{tabela}'")


if __name__ == "__main__":
    if not MONGO_URI:
        raise SystemExit("Configure MONGO_URI no arquivo .env para acessar o MongoDB.")
    if not NEON_DATABASE_URL:
        raise SystemExit(
            "Configure NEON_DATABASE_URL no arquivo .env para acessar o Neon."
        )

    cliente_mongo = MongoClient(MONGO_URI)
    banco_mongo = cliente_mongo[NOME_BANCO_MONGO]

    engine_neon = create_engine(NEON_DATABASE_URL)

    criar_tabelas(engine_neon)

    print("Transformando unidades_saude...")
    inserir_linhas(
        engine_neon,
```

```
        "unidades_saude",
        transformar_unidades_saude(banco_mongo["cnes_estabelecimentos_detalhe"]),
    )

    print("Transformando equipes_saude...")
    inserir_linhas(
        engine_neon,
        "equipes_saude",
        transformar_equipes_saude(banco_mongo["cnes_equipes"]),
    )

    print("Transformando censo_raca_cor...")
    inserir_linhas(
        engine_neon,
        "censo_raca_cor",
        transformar_censo_raca_cor(banco_mongo["ibge_censo_raca_cor"]),
    )

    print("Transformando censo_deficiencia...")
    inserir_linhas(
        engine_neon,
        "censo_deficiencia",
        transformar_censo_deficiencia(banco_mongo["ibge_censo_deficiencia_tipo"]),
    )

    print("\nConcluído!")
```

04 | Inserção dos dados simulados

Script Python completo: MongoDB → PostgreSQL/Neon, tabelas mock_.

Arquivo de origem: `src\transform\mongo_para_neon_mock.py`

```
"""
Transforma os dados FICTÍCIOS/SIMULADOS de camada bronze (MongoDB) em tabelas
relacionais na camada silver (Neon/PostgreSQL).

Não representa pacientes, atendimentos ou população real de Recife — ver aviso
nas próprias tabelas (COMMENT ON TABLE) e nos documentos de origem (_meta.aviso).
"""

import os

from dotenv import load_dotenv
from pymongo import MongoClient
from sqlalchemy import create_engine, text

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
NEON_DATABASE_URL = os.getenv("NEON_DATABASE_URL")
NOME_BANCO_MONGO = "bronze_equidade_saude"

DDL_TABELAS = """
CREATE TABLE IF NOT EXISTS mock_pec_atendimentos (
    id_atendimento                      INTEGER PRIMARY KEY,
    codigo_equipe                       TEXT REFERENCES equipes_saude(codigo_equipe),
    cnes_unidade                         TEXT REFERENCES unidades_saude(cnes),
    distrito_sanitario_codigo            TEXT,
    data_atendimento                    DATE,
    tipo_atendimento                    TEXT,
    profissional_tipo                   TEXT,
    turno_atendimento                   TEXT,
    idade                                INTEGER,
    raca_cor                            TEXT,
    deficiencia_tipo                    TEXT,
    deseja_informar_orientacao_sexual   TEXT,
    orientacao_sexual                   TEXT,
    deseja_informar_identidade_genero   TEXT,
    identidade_genero                   TEXT,
    nome_social                         TEXT
);
ALTER TABLE mock_pec_atendimentos
    ADD COLUMN IF NOT EXISTS turno_atendimento TEXT;
COMMENT ON TABLE mock_pec_atendimentos IS
    'DADOS FICTICIOS/SIMULADOS -- nao representam pacientes ou atendimentos '
    'reais. Gerados para testar o pipeline enquanto o acesso ao PEC/e-SUS real '
    'nao e liberado pela Secretaria.';

CREATE TABLE IF NOT EXISTS mock_populacao_negra_distrito (
    distrito_sanitario_codigo      TEXT PRIMARY KEY,
    populacao_negra_pct_esperado   NUMERIC
);
COMMENT ON TABLE mock_populacao_negra_distrito IS
    'DADOS FICTICIOS/SIMULADOS -- distribuicao ilustrativa da populacao negra '
    'por Distrito Sanitario, usada como referencia ("esperado") para comparar '
    'com o observado em mock_pec_atendimentos. Nao representa a distribuicao '
    'real de Recife.';
"""


def criar_tabelas(engine):
    with engine.begin() as conexao:
        conexao.execute(text(DDL_TABELAS))
    print("Tabelas mock criadas.")


def transformar_mock_atendimentos(colecao):
    linhas = []
    for doc in colecao.find():
        linhas.append({
            "id_atendimento": doc.get("id_atendimento"),
            "codigo_equipe": doc.get("codigo_equipe"),
            "cnes_unidade": doc.get("cnes_unidade"),
            "distrito_sanitario_codigo": doc.get("distrito_sanitario_codigo"),
            "data_atendimento": doc.get("data_atendimento"),
            "tipo_atendimento": doc.get("tipo_atendimento"),
            "profissional_tipo": doc.get("profissional_tipo"),
            "turno_atendimento": doc.get("turno_atendimento"),
```

```
            "idade": doc.get("idade"),
            "raca_cor": doc.get("raca_cor"),
            "deficiencia_tipo": doc.get("deficiencia_tipo"),
            "deseja_informar_orientacao_sexual": doc.get("deseja_informar_orientacao_sexual"),
            "orientacao_sexual": doc.get("orientacao_sexual"),
            "deseja_informar_identidade_genero": doc.get("deseja_informar_identidade_genero"),
            "identidade_genero": doc.get("identidade_genero"),
            "nome_social": doc.get("nome_social"),
        })
    return linhas


def transformar_mock_populacao_negra_distrito(colecao):
    linhas = []
    for doc in colecao.find():
        linhas.append({
            "distrito_sanitario_codigo": doc.get("distrito_sanitario_codigo"),
            "populacao_negra_pct_esperado": doc.get("populacao_negra_pct_esperado"),
        })
    return linhas


def inserir_linhas(engine, tabela, linhas, tamanho_lote=1000):
    if not linhas:
        print(f"  nenhuma linha para inserir em '{tabela}'")
        return
    colunas = list(linhas[0].keys())
    total_inserido = 0
    with engine.begin() as conexao:
        for inicio in range(0, len(linhas), tamanho_lote):
            lote = linhas[inicio:inicio + tamanho_lote]
            valores_sql = []
            parametros = {}
            for i, linha in enumerate(lote):
                nomes_params = [f"{coluna}_{i}" for coluna in colunas]
                valores_sql.append("(" + ", ".join(f":{n}" for n in nomes_params) + ")")
                for coluna, nome_param in zip(colunas, nomes_params):
                    parametros[nome_param] = linha[coluna]
            atualizacoes = ", ".join(
                f"{coluna} = EXCLUDED.{coluna}" for coluna in colunas[1:]
            )
            sql = text(
                f"INSERT INTO {tabela} ({', '.join(colunas)}) VALUES "
                f"{', '.join(valores_sql)} "
                f"ON CONFLICT ({colunas[0]}) DO UPDATE SET {atualizacoes}"
            )
            conexao.execute(sql, parametros)
            total_inserido += len(lote)
    print(f"  {total_inserido} linha(s) inserida(s) em '{tabela}'")


if __name__ == "__main__":
    if not MONGO_URI:
        raise SystemExit("Configure MONGO_URI no arquivo .env para acessar o MongoDB.")
    if not NEON_DATABASE_URL:
        raise SystemExit(
            "Configure NEON_DATABASE_URL no arquivo .env para acessar o Neon."
        )

    cliente_mongo = MongoClient(MONGO_URI)
    banco_mongo = cliente_mongo[NOME_BANCO_MONGO]

    engine_neon = create_engine(NEON_DATABASE_URL)

    criar_tabelas(engine_neon)

    print("Transformando mock_pec_atendimentos...")
    inserir_linhas(
        engine_neon,
        "mock_pec_atendimentos",
        transformar_mock_atendimentos(banco_mongo["mock_pec_atendimentos"]),
    )

    print("Transformando mock_populacao_negra_distrito...")
    inserir_linhas(
        engine_neon,
        "mock_populacao_negra_distrito",
        transformar_mock_populacao_negra_distrito(banco_mongo["mock_populacao_negra_distrito"]),
    )

    print("\nConcluído! Lembre-se: essas duas tabelas contêm dados fictícios/simulados.")
```